

全国大学生计算机系统能力大赛操作系统赛
决赛产品文档

AgentOS-uCore

系统原生智能体运行时

基于 RISC-V uCore 的身份、工具、上下文、调度与协作机制

作品名称: AgentOS-uCore
交互平台: AgentOS Console / AgentOS Nexus
目标平台: RISC-V 64 / uCore / QEMU
文档版本: 决赛版

摘要

现有智能体平台通常在应用层组织模型、工具和多轮对话，而操作系统仍将其视为普通进程。随着 Agent 开始长时间运行、跨角色协作并产生文件和系统副作用，身份、权限、资源和因果关系仅依靠 prompt 或应用日志已难以支撑稳定的系统约束。

我们基于 LearningOS/uCore 实现了 AgentOS-uCore。产品以 uCore 的进程、虚拟内存、VFS、系统调用和调度器为底座，向上增加六组相互衔接的系统机制：内核原生 Agent 身份与生命周期、结构化工具执行、Context Path 与数据来源传播、Live-Query 文件系统、事件驱动 Agent Loop 与 Workflow EEVDF，以及跨模块的综合运行场景。这些机制使 Agent 成为内核可识别、可授权、可挂起、可记账和可调度的工作负载。

在内核机制之上，AgentOS Console 提供确定性科研恢复工作流，AgentOS Nexus 组织 Coordinator、System、Research 和 Analyst 四个长驻业务 Agent。Nexus 使用统一的子任务协议和工件数据面完成动态委派，并在同一个 QEMU boot 中展示身份、Context、文件、IPC、等待与调度状态。

为了分析内核数据路径，我们进行了一次性增强实验，完成 30 次 fresh QEMU boot，保存 33 份原始输出、19 个 CSV 数据表和 7,498 条逐样本记录。16 个独立 workflow 配对中，indexed core path 全部取得更低延迟，配对加速比中位数为 3.118 倍。文件查询网格覆盖 12 组 catalog size 与 hit count 组合，每组保留 15 个 AB/BA inner pair。Task 实验保留 96 条 16-op sequence；EEVDF 实验保留 504 条精确唤醒探针，全部在 0–1 tick 内得到服务。

核心功能

表 1: AgentOS-uCore 核心功能完成情况

序号	功能模块	实现内容	验证规模
1	身份与生命周期	受信映像创建、role/capability、workflow id+generation、七页 Context 地址区、资源账户	真实 Guest 场景
2	结构化工具	25 项工具目录、typed V2、ENFORCE V3、compact batch、Task SQ/CQ	96 条 sequence
3	Context 与运行记录	128 条 Context Path、provenance、Evidence Ring、Workflow Fence	64 轮与淘汰场景
4	Live-Query FS	boot-scoped metadata、选择性索引、typed watch、resync generation	180 个文件配对
5	Loop 与调度	watch/wait、heartbeat、MESSAGE route、LLM correlation、Workflow EEVDF	504 条 wake probe
6	综合产品场景	AgentOS Console、AgentOS Nexus、四角色协作、双窗口交互	QEMU 端到端运行

目录

摘要	ii
第一章 项目背景与设计目标	1
1.1 Agent 对操作系统提出的新需求	1
1.2 基于 uCore 的系统扩展	1
1.3 机制与策略的分层	2
1.4 相关机制与技术选择	2
第二章 AgentOS-uCore 总体架构	3
2.1 以 uCore 为底座的产品分层	3
2.2 六项核心机制	4
2.3 一次 Agent 工具调用	4
2.4 上层运行时	4
第三章 内核原生 Agent 身份与生命周期	6
3.1 从进程身份到 workflow 主体	6
3.2 身份字段与角色策略	6
3.3 七页 Context 地址区	6
3.4 Lifecycle generation 与并发 gate	7
3.5 资源主体随 workflow 创建	7
3.6 验证结果	7
第四章 结构化工具执行	8
4.1 工具目录与参数模式	8
4.2 Exploratory typed V2	8
4.3 四种执行路径	8
4.4 Tool Phase Credit Lease	9
4.5 Typed Task Channel	9
4.6 验证结果	9
第五章 Context Path、Provenance 与 Workflow Fence	10
5.1 从多轮文本到结构化 Context	10
5.2 分支、回退与淘汰	10
5.3 数据来源传播	10
5.4 工具 Manifest 与效果检查	11

5.5	Evidence Ring	11
5.6	Workflow Fence	11
5.7	验证结果	12
第六章	Agent Live-Query 文件系统	13
6.1	Agent 对文件对象的结构化需求	13
6.2	Metadata 与 inode incarnation	13
6.3	Traversal 与选择性索引	13
6.4	Typed live watch 与世代恢复	13
6.5	查询性能	14
6.6	验证结果	15
第七章	Agent Loop 与 Workflow EEVDF	16
7.1	从轮询循环到内核等待	16
7.2	Heartbeat 与 LLM correlation	16
7.3	Workflow Credit Domain	16
7.4	按 workflow 聚合的 EEVDF	17
7.5	唤醒延迟与公平性	17
7.6	验证结果	17
第八章	AgentOS Console 综合工作流	18
8.1	科研恢复场景	18
8.2	Traversal 与 Indexed 配对	18
8.3	Core 与端到端窗口	19
8.4	系统结果	19
第九章	AgentOS Nexus 多智能体协作平台	20
9.1	长驻会话与四角色协作	20
9.2	一次交互回合	20
9.3	N1 子任务协议	21
9.4	Artifact 数据面	21
9.5	动态工具呈现	22
9.6	调用级审批与取消	22
9.7	内核观测面	22
9.8	运行验证	23
第十章	功能与性能测试	24
10.1	测试组织	24
10.2	一次性增强实验	24

10.3 Task 传输延迟	25
10.4 内核 I/O 与工作量	26
10.5 可重复性	26
第十一章 总结与展望	27

插图

2.1	AgentOS-uCore 产品架构	3
2.2	Plain uCore 与 AgentOS-uCore 内核路径	5
6.1	Catalog 查询加速比	14
6.2	Traversal 与 Indexed 查询分组	14
7.1	EEVDF 唤醒延迟与公平性	17
8.1	Traversal 与 Indexed Core 配对性能	18
8.2	Core 与端到端窗口	19
9.1	Nexus 四角色委派与三平面运行时	21
10.1	Task 延迟分布	25
10.2	内核 I/O 与工作量	26

表格

1	AgentOS-uCore 核心功能完成情况	ii
1.1	uCore 基础与 AgentOS-uCore 增量	1
1.2	AgentOS-uCore 的三级职责	2
2.1	AgentOS 核心模块	4
3.1	Agent 身份核心字段	6
4.1	工具执行路径	8
5.1	Context provenance 标签	11
9.1	Nexus 业务角色	20
10.1	六项核心机制的测试场景	24
10.2	一次性实验组成	25

项目背景与设计目标

1.1 Agent 对操作系统提出的新需求

大模型工具调用已经从单次问答发展为持续运行的 Agent Loop。一个典型工作流会反复执行“读取上下文、选择工具、等待结果、修正计划”，并将任务拆分给多个专职 Agent。这种负载具有三个突出特点：生命周期长，工具副作用丰富，协作中间状态多。

传统进程抽象保存 PID、内存、文件和 CPU 时间，却无法直接表达 Agent 的角色、工作流世代、上下文因果、工具模式和数据来源。这些信息若只保存在 prompt、JSON 和应用日志中，文件、IPC 与资源操作就难以在实际发生前得到统一检查。一个角色还可以通过创建更多线程放大调度份额，进而影响其他工作流的服务时间。

我们将 AgentOS-uCore 的目标归纳为一句话：**让操作系统识别 Agent 工作负载，并在系统效果发生处提供一致的身份、资源、因果和调度机制。**

1.2 基于 uCore 的系统扩展

项目建立在 LearningOS/uCore 教学内核之上 [1]。uCore 提供 RISC-V 启动、进程、虚拟内存、VFS、trap、系统调用和基础调度机制。我们沿用这些主干，在现有 PCB、VFS 对象、调度队列和 syscall 路径上增加 AgentOS 状态。所有新 UAPI 使用定长结构、显式 version/size 和 kernel/user 共用布局检查。普通 uCore 进程继续沿用原有路径，Agent 进程则在受控的创建和 exec 路径上获得内核身份。

表 1.1: uCore 基础与 AgentOS-uCore 增量

系统对象	uCore 基础	AgentOS-uCore 增量
进程与线程	PID、内存、文件表、运行状态	Agent identity、role/capability、control id、workflow lifecycle
虚拟内存	页表、用户页与内核页	六页只读 Context mirror 与一页用户 cache
文件系统	inode、目录、读写与 fsync	boot-scoped metadata、选择性索引、typed live query
系统调用	参数拷贝、执行与返回	结构化工具目录、typed V2/V3、Task SQ/CQ
进程间通信	管道、信号与基本等待	MESSAGE route、watch/wait、heartbeat、LLM correlation
调度与资源	线程运行队列	workflow 实体、Credit Domain、EEVDF lag/deadline

文件 metadata 使用当前 boot 的内存 sidecar，并与真实 dev+inum+incarnation 绑定。这种实现方式保留原 uCore 磁盘格式，也使索引、订阅和世代恢复能够独立演进。

1.3 机制与策略的分层

我们把稳定、有界、与系统效果直接相关的机制放入内核；将目标分解、历史组织和工具选择留在 Guest 用户态；将 TLS、API key 和 provider 协议放在 Host。这样的层次使确定性策略、strict replay 和真实模型共用同一套 Guest 与内核路径。

表 1.2: AgentOS-uCore 的三级职责

层次	主要对象	职责
Host	CLI、observer、provider adapter、QEMU serial	交互、传输、TLS 和 provider 格式转换
Guest 用户态	Coordinator、specialists、N1、artifact store	目标分解、任务状态机、工件组织和结果回灌
AgentOS 内核	identity、Context、tool、VFS、IPC、resource、scheduler	身份、权限、数据来源、等待、记账与调度

1.4 相关机制与技术选择

资源记账参考 Linux cpuacct/rstat 的分层聚合思路 [2, 3]，运行记录参考 BPF ring buffer 的有序提交 [4]，调度使用 EEVDF 的 lag 与 virtual deadline 概念 [5]。Task Channel 采用 io_uring 的 SQ/CQ 通信形状 [6]，文件 metadata 和选择性索引参考 Haiku BFS[7]。Agent 工具循环中“模型选择、应用执行、结果回灌”的职责分配来自现有 tool-use 实践 [8, 9]。

这些选择在 uCore 中被重新组织为统一的 workflow 主体。identity、Context、VFS scope、Credit Domain 与 EEVDF 共用同一个 id+generation，从而避免各模块对 Agent 归属产生不同理解。

AgentOS-uCore 总体架构

2.1 以 uCore 为底座的产品分层

AgentOS-uCore 是一组直接运行在 uCore 内核中的 Agent 功能模块。图2.1从下到上给出产品结构：uCore 提供基础系统对象，AgentOS 将六组核心机制嵌入这些对象，Console 与 Nexus 则使用统一 UAPI 组织单 Agent 和多 Agent 工作流。

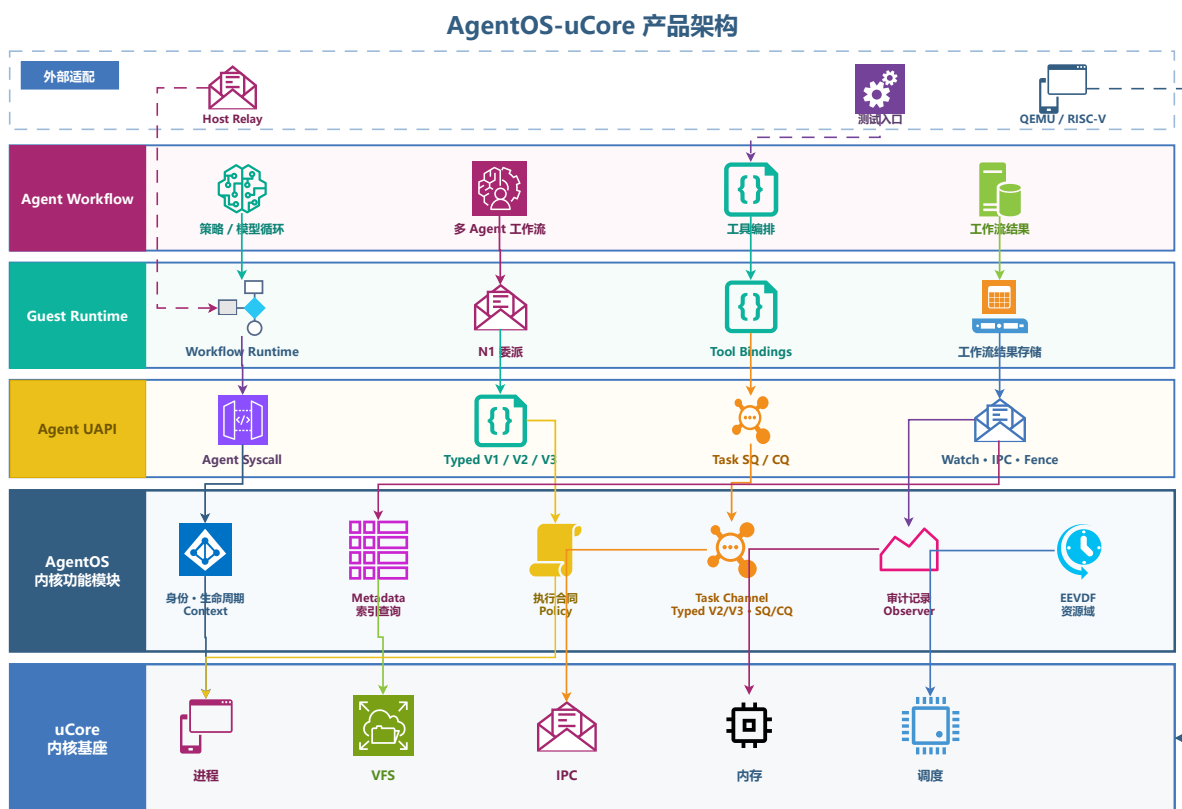


图 2.1: AgentOS-uCore 产品架构

底层 uCore 主干保持原有的进程、页表、VFS、trap、syscall 和 scheduler 责任。AgentOS 在其上形成三条相互配合的纵向路径：

1. **运行主体路径**：identity 和 lifecycle 创建 Agent 主体，Context 保存每次工具交互，Credit Domain 与 EEVDF 负责资源和 CPU 服务。
2. **系统效果路径**：typed tool 对调用进行 schema/capability 检查，VFS metadata 和 live query 处理文件对象，MESSAGE route 处理跨 Agent 通信。
3. **运行记录路径**：Context Path 记录因果链，provenance 随文件、工具和 IPC 传播，Evidence

Ring 与 Workflow Fence 产生工作流级截面。

2.2 六项核心机制

表 2.1: AgentOS 核心模块

模块	关键对象	主要功能
身份与生命周期	identity、control id、lifecycle	创建受信 Agent，绑定 role/capability、workflow generation 与 VFS scope
结构化工具	catalog、V2/V3、SQ/CQ	发现工具，检查 typed 参数，执行适应式调用、冻结合同和有界任务提交
Context 与 Provenance	Context Path、mirror、ring	连接请求、结果、因果、分支和数据来源
Live-Query FS	metadata、index、watch	为显式登记文件提供属性、选择性查询、变化通知和世代恢复
Agent Loop	event queue、route、heartbeat	提供原子 watch/wait、MESSAGE、LLM correlation、timeout 和 cancel
Workflow 资源与调度	Credit Domain、EEVDF entity	按 workflow 聚合记账，使用 lag 和 virtual deadline 分配 CPU 服务

六项机制通过统一 workflow id+generation 相互关联。一次工具调用同时关联调用者身份、Context predecessor、工具 schema、数据 provenance、资源账户与调度实体。workflow generation 使已经回收的 slot、watch 和 handle 不会命中新对象。

2.3 一次 Agent 工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 根据 role 构建可见工具集，将模型结果编码为 typed request。请求进入内核后，AgentOS 依次完成 copyin 与 version/size 检查、identity/lifecycle 检查、tool resolve 与 schema decode，再执行 contract、provenance、resource 和 effect 检查，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，Context Path 发布新记录，下一轮可直接引用其 sequence。

当 Agent 等待模型、文件事件或其他角色时，agent_wait 将线程移入等待状态。事件入队与 waiter 安装使用原子顺序，从而避免“查看队列为空后刚好错过唤醒”。

2.4 上层运行时

Console 使用确定性策略连接身份、Context、文件查询、IPC 与恢复操作，用于功能演示和配对测量。Nexus 在同一系统调用面上增加长驻会话、动态委派、工件组织和 provider 对接。两个运行时共用 AgentOS 内核对象，可以在确定性测试和自然语言交互之间保持一致的系统语义。

图2.2从运行时序角度展示同一科研恢复任务的两种实现。Plain uCore 使用普通进程、文件与 IPC；AgentOS-uCore 在这条路径中加入可识别的 workflow 主体、结构化工具、选

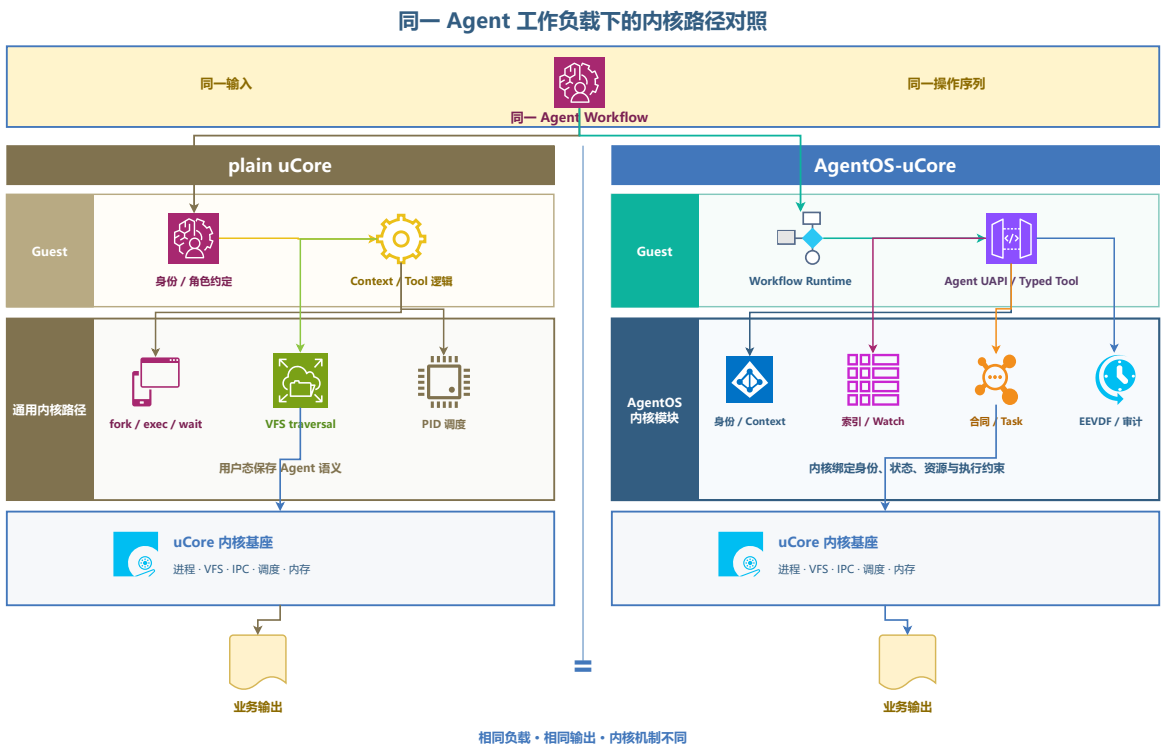


图 2.2: Plain uCore 与 AgentOS-uCore 内核路径

择性索引、事件等待和资源调度，使每个阶段都能关联到明确的 Agent 与 workflow。

内核原生 Agent 身份与生命周期

3.1 从进程身份到 workflow 主体

AgentOS 需要区分普通进程、编排者和专职 Agent，同时让这些身份可以被 VFS、IPC、工具和调度器共用。我们将 Agent 身份作为内核状态安装到 PCB，并通过受信映像 manifest 和父角色授权创建。用户态程序可以读取 identity，权限判定始终使用内核副本。

创建过程是一个完整事务：内核首先分配 workflow lifecycle，安装 identity、role/capability 和 control id，然后建立 Context 页、VFS scope 和资源账户。任一步骤失败都回收已分配对象。创建完成后，各子系统通过同一 lifecycle key 找到该 Agent 所属的工作流。

3.2 身份字段与角色策略

表 3.1: Agent 身份核心字段

字段	含义	使用模块
agent_id	当前 boot 内的 Agent 编号	工具、Context、观测
control_id	内核分配的控制身份	controller 绑定、IPC route、handle 校验
role/capability	角色及其具体权限集	工具、文件、消息与工件效果
workflow id+generation	工作流标识与可复用 slot 的世代	lifecycle、watch、Task、resource、scheduler
VFS scope	工作流文件可见域	metadata、query、artifact
exec/storage account	工作流资源归属	进程、线程、文件、块、页与缓冲区

内核 role policy 将角色映射为 capability mask 和调度权重。Sentinel 主要读取进程、metadata 与调度状态；Investigator 获得受限内容读权限；Artifact 角色可以创建报告工件；Orchestrator 持有编排和角色创建权限。子 Agent 继承同一 workflow scope，其 capability 受父角色授权和映像 manifest 共同约束。

3.3 七页 Context 地址区

每个 Agent 拥有固定 ABI 地址的七页 Context。前六页由内核维护，Guest 页表映射为只读，用于发布 identity header、最新返回、Context records、路径索引和稳定摘要。第七页

是用户 cache, Guest 用于保存高频策略数据。identity、capability、scope 与因果序列均来自前六页的内核状态。

Context 固定容量为 128 条。每条记录包含 sequence、request/tool/status、service-start tick、cause、span、branch generation、control id 与记录摘要。内核使用 publish sequence 更新 mirror, Guest 也可以通过 query/snapshot UAPI 取得相同结构。

3.4 Lifecycle generation 与并发 gate

workflow slot 可在回收后复用, generation 每次重新分配。watch、资源账户、VFS sidecar、Task handle 与 artifact 都同时保存 id 和 generation。

lifecycle 使用 operation、departure 和 fence 三类 gate。普通系统调用进入 operation gate, exit/teardown 进入 departure gate, Workflow Fence 在两类操作排空后形成截面。当 controller 关闭且 member、active operation 和 departure 都归零时, 内核回收 Context、watch、VFS scope、ring 和资源账户。

3.5 资源主体随 workflow 创建

workflow admission 同时分配 exec 和 storage account。工作流内的进程、线程、文件对象、文件系统块/inode、Agent state page 和物理页均记录在同一账户下。这一设计使后续 Phase Lease 和 Workflow EEVDF 可以按工作流统一计算, 子进程和多线程仍属于原工作流。

3.6 验证结果

agenttrust_ucore 覆盖受信 exec、role grant、capability 衰减、fork/exec/exit 与普通进程共存; agentscope_ucore 覆盖 scope 与 generation 复用; agentfinal_ucore 覆盖 Context 固定映射、snapshot、branch、rollback 和容量淘汰。这些场景都在 RV64 uCore Guest 中执行真实系统调用。

结构化工具执行

4.1 工具目录与参数模式

Agent 对工具的需求同时包含“找到可用能力”和“在效果发生前校验调用”。AgentOS 内核维护 25 项工具目录，每项 descriptor 包含 version、size、tool id、name、schema、flags 与 side-effect 类别。启动阶段会检查 id/name 唯一性和 schema 完整性。

`sys_tool_list` 向 Guest 返回定长 descriptor。Guest runtime 再结合 role/capability 构建模型可见的工具集，补充用途、参数和结果说明。运行时内部的 wait、Context 与 LLM 原语保留给 Guest 自身，模型只看到当前角色可以请求的产品动作。

4.2 Exploratory typed V2

V2 request 携带显式 version、request size、tool name/id、parameter count 和 sized typed parameter 数组。当前 scalar 类型为 uint64 与 string，适合 uCore 中紧凑的参数解析。内核一次性拷贝参数，完成 key、type、数量和长度检查后，再进入共享执行器。

V2 适合自适应 Agent Loop。例如 Coordinator 可以先查询失败 stage，根据候选数决定是读取摘要还是创建 Research 子任务。每次调用独立检查 identity、capability、scope、schema、provenance 与资源状态。

4.3 四种执行路径

表 4.1: 工具执行路径

路径	输入形式	主要用途	运行语义
Compact batch Typed V2	最多 64 个 agent_op typed key/value	固定顺序与 Context 压力 场景 根据上一轮结果选择工具	一次 syscall 内顺序执行， 逐项返回状态 逐次 schema/capability/scope 检查
ENFORCE V3	V2 prefix 与 contract envelope	可预先定义的高保证 DAG	冻结 node、predecessor、 attempt、deadline 与 effect manifest
Task SQ/CQ	16-slot SQ 与 16-slot CQ	有界提交、完成与 backpressure	单 issuer、世代校验、每 请求一个 terminal CQE

Compact batch 用于同步小操作序列。V3 则在 workflow lifecycle 内冻结一份版本化合同，最多包含 24 个拓扑有序节点。每个节点关联 tool、schema digest、required capability、input/output artifact type、deadline、attempt、provenance/effect manifest 与 exec/storage envelope。

V3 request 同时绑定 contract generation、node/attempt、source node、predecessor Context sequence 和 32-byte fingerprint。成功或失败的 terminal 结果进入稳定完成槽，合法重试直接读取原结果，避免重复产生副作用。

4.4 Tool Phase Credit Lease

高风险工具可在执行前从 workflow account 锁定 exec/storage envelope。Phase Lease 依次经历 ADMITTED-→ACTIVE-→DEACTIVATED-→SETTLED。分配器在发布对象前获得 nonce claim，成功对象继续占有同一份 used credit，析构时完成一次结算。这一状态机适合多种对象在一次工具中共同达到峰值的场景。

4.5 Typed Task Channel

Task Channel 为每个 Agent 按需映射 16-slot SQ 和 16-slot CQ，请求与资源状态保存在两个内核私有页中。SQE/CQE 均为 128 字节。内核先拷贝完整 SQE，再检查 channel generation、递增 request id、contract/node/attempt、schema、deadline、cancel link 和 typed handle。channel generation 不匹配时返回 STALE。

内核私有 head/tail 是权威状态。CQ 满时形成 backpressure；SQE ring/slot generation 不一致或其他协议故障会使通道进入 sticky resync，issuer 显式恢复后继续提交。每个 accepted target 在 success、failure、cancel 或 timeout 中发布一个 terminal CQE。当前 provider 接受 null input 并返回 NONE，用于验证通道、截止时间、取消和完成语义。

4.6 验证结果

agenttoolabi_ucore 覆盖目录发现、name/id 一致性、未知参数、类型错误和返回缓冲区；agentcontract_ucore 覆盖 DAG、deadline、retry/cancel、source Context 与 provenance gate；agenttask_ucore 覆盖 SQ/CQ full、resync、cancel 和 terminal retention。一次性数据集进一步保存 96 条 16-op sequence，其分布在第十章集中分析。

Context Path、Provenance 与 Workflow Fence

5.1 从多轮文本到结构化 Context

Agent 的下一步决策通常依赖上一次工具结果。AgentOS 在每次工具执行的效果位置建立 Context record，将请求、结果、因果和数据来源组织为可查询路径。

记录包含递增 sequence、request/tool/status、service-start tick、cause sequence、span、branch generation、control id、短 payload/result 和 record hash。path_parent_sequence 指向当前 active head，successor index 支持分支查询。

Context append 使用 prepare、fill、publish、commit 四个阶段。mirror header 发布 count、head、oldest、latest、visible head、dropped 和 rollback count。Guest 在读取记录后重验 sequence/hash，再次确认 summary 未变，从而获得一致快照。

5.2 分支、回退与淘汰

Context Path 保留 128 条记录。容量用尽后按 sequence 淘汰最早记录，同时更新 oldest/latest 和 dropped 计数。active path 只引用当前保留的 predecessor。

rollback 将 visible_head 移到一个仍然保留的 Context sequence，并为后续调用分配新 branch generation。已产生的文件、IPC 和 provider 效果保留原状。需要幂等的效果由 V3 attempt cache 与具体工具协议控制。这使 Context 分支可以专注于后续决策视图。

5.3 数据来源传播

AgentOS 使用六类可组合 provenance 标签，标签随 Context、文件、工具返回、IPC 和 artifact 传播。

文件、工具、mailbox 和 Task resource 使用保守 OR 传播。摘要、重新哈希和跨 Agent materialize 会增加新来源，同时保留原标签。因此，文件数据可以作为模型观测，但不会自动成为控制授权。

表 5.1: Context provenance 标签

标签	含义	典型来源
KERNEL_FACT	内核结构化事实	PID、resource、scheduler、capability
TRUSTED_USER_CONTROL	经产品控制面提交的用户控制	新 turn、参数绑定审批
AGENT_DERIVED	Agent 从已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DATA	来自文件的内容或 metadata	query、read、digest
UNTRUSTED_TOOL_OUTPUT	工具执行返回	typed V2/V3 result
CROSS_AGENT_DATA	经 IPC 或 artifact 跨角色传递	TASK result、broker artifact

5.4 工具 Manifest 与效果检查

每个内核工具包含 safety manifest, 其字段包括 accepted input labels、output-added labels、required capability 与 file/metadata/IPC/process/permission/artifact/watch side-effect mask。启用 ENFORCE V3 后, lifecycle generation、frozen contract edge、schema、capability、manifest 和 provenance 必须同时匹配。失败请求在工具效果前返回结构化状态, 并记录 source Context、tool 与 reason。

5.5 Evidence Ring

每个活跃 workflow 按需分配四个 Agent state page: 三页 ordinary ring, 共 48 个 256-byte slot; 一页 critical ring, 共 16 个 slot。生产者预留递增 ticket, 填充后提交。当更早 ticket 仍在准备中时, 读者暂停在当前位置, 从而保持全局发布顺序。

普通成功 Context 写入 ordinary ring, 授权效果和拒绝写入 critical ring。环形空间复用前, 已提交 segment 滚入内部 root。这套结构为 Workflow Fence 提供按 ticket 排序的运行记录。

5.6 Workflow Fence

controller 通过 `agent_run(count=0, AGENT_RUN_F_FENCE)` 发起 fence。内核检查 orchestrator/control 关系, 取得 lifecycle fence gate, 排空 metadata/live-query 延迟工作和 VFS reclaim, 紧缩 exec/storage credit 并确认 pending credit 归零, 最后密封 Evidence Ring。

320-byte receipt 绑定 previous root、challenge、fence sequence、ticket range、event/gap count、metadata generation、credit epoch、精确 used-credit digest 和 ordered segment digest。receipt 在内部状态提交后 copyout, 同一 fence id 的 copyout 重试返回已缓存结果。该 receipt 表达当前 boot 内的 workflow 截面, 其 flags 同时记录 metadata 的 volatile 性质与 coverage 状态。

5.7 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，然后覆盖 `snapshot`、`branch`、`rollback` 和窗口淘汰。`provenance` 场景将文件来源传入受保护副作用，验证效果前拒绝与 `critical ring` 记录。`fence` 场景覆盖 `challenge`、`retry cache`、`credit pending`、`metadata generation` 和 `copyout` 失败重试。

Agent Live-Query 文件系统

6.1 Agent 对文件对象的结构化需求

科研和恢复 workflows 经常围绕“当前哪些对象满足条件”与“哪些对象刚刚发生变化”运行。目录遍历可以找到路径，workflows 还需要 project、run、stage、status、kind、dependency 和内容摘要等属性。AgentOS 在 uCore VFS 之上增加 boot-scoped metadata catalog，将显式登记的文件组织为可查询、可订阅的 Agent 对象。

6.2 Metadata 与 inode incarnation

拥有 META_WRITE 的 Agent 调用 `agent_file_meta_set()` 设置 metadata，字段与真实 dev+inum+incarnation 绑定。更新使用 `update_mask`，删除使用显式 DELETE 动作。文件 create/rename 继续使用原 uCore VFS，只有 workflows 明确关心的对象进入 catalog。

incarnation 区分 inode number 复用前后的两个对象，并由 VFS 增量维护，随 query/watch 结果返回。query 或 watch 命中记录后，内核复核 catalog generation、lifecycle、scope 与完整 predicate。unlink、内容大小变化与对象失效会更新已登记 sidecar。

6.3 Traversal 与选择性索引

catalog 为 status/stage/kind 维护选择性索引。typed query 包含 predicate、scope、plan 和最大 hit 数，返回结构化对象、candidate count、records examined、result count、generation 与 cache state。调用者可强制 traversal，也可请求 index plan。索引路径仍对候选记录执行完整 predicate 检查，保证 traversal/indexed 结果一致。最大返回数为 8，使查询缓冲区有明确上限。

6.4 Typed live watch 与世代恢复

`agent_live_watch()` 安装一份完整 `agent_file_query`。每次 metadata 变化计算 before/after predicate：从未命中变为命中产生 ENTER，持续命中且记录变化产生 UPDATE，从命中变为未命中或对象删除产生 LEAVE。事件通过 AGENT_EVENT_FILE_QUERY 进入目标 Agent 的 Context 和有界队列。

当队列或 pending 表无法保存全部增量时，内核递增 `resync_generation` 并发送 RESYNC_REQUIRED。Agent 重新执行 snapshot/query，然后携带对应 generation 调用

ACK_RESYNC。Workflow Fence 会排空当前 scope 的 unlink/content pending，并在 resync 完成后密封 metadata generation。

6.5 查询性能

实验使用 catalog size 24、64、96 和 exact hit count 1、2、4、8，组成 12 个实测网格。每格保留 15 个 AB/BA inner pair，同时记录 traversal/indexed latency、candidate count 和 records examined。

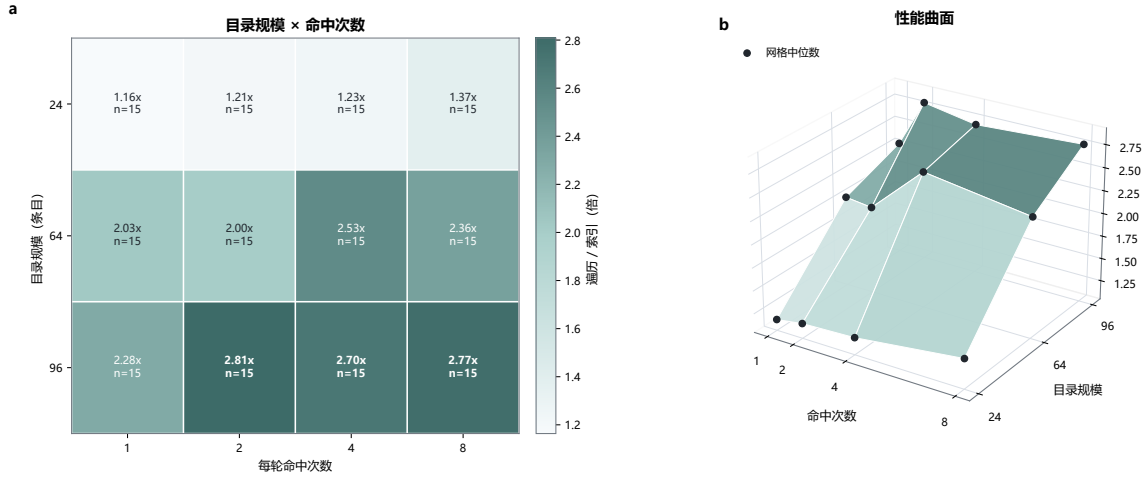


图 6.1: Catalog 查询加速比

图6.1的 12 个格点均来自实测，中位加速比范围为 1.164–2.808 倍。catalog 越大、hit 越集中时，选择性索引能够跳过更多无关记录。三维曲面用于连续展示这一趋势，热力图中的每个数字则对应原始网格样本。

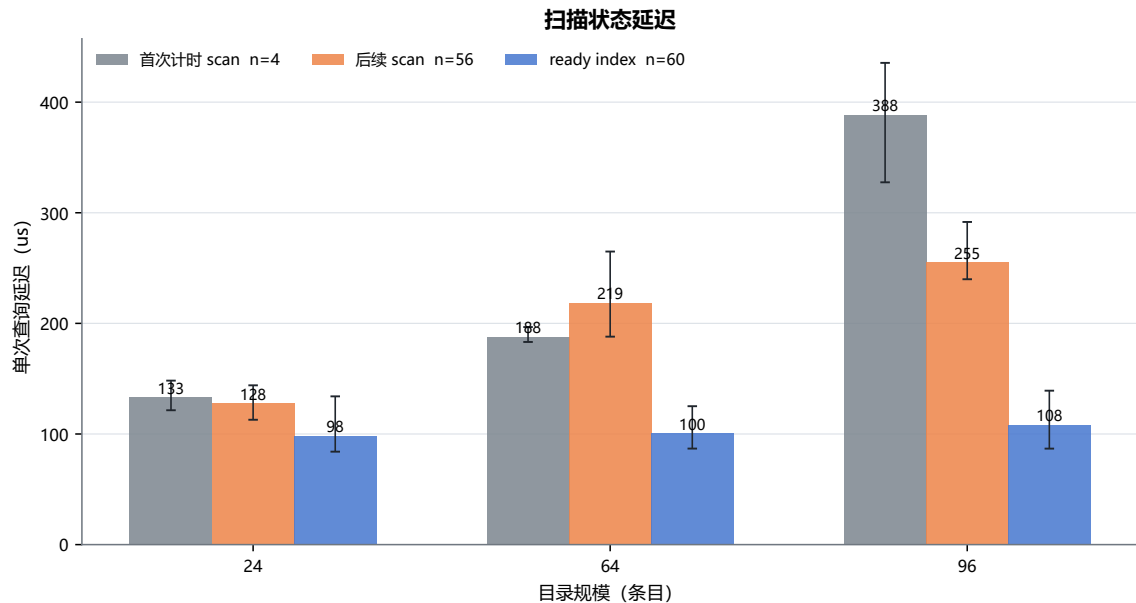


图 6.2: Traversal 与 Indexed 查询分组

图6.2将显式 warmup 后的第一次 timed scan、后续 repeat scan 和 ready index 分组展示。

28 条 diagnostic 均记录 `cache=ready`，因此该图聚焦查询计划差异，而非宿主页缓存的冷启动效应。

6.6 验证结果

`agentfs_ucore` 覆盖 `metadata set/update/delete`、`summary/digest`、`incarnation` 和 `scope`；`agentbench_ucore` 用同一数据集比较 `traversal/indexed` 的结果集、候选数与延迟。Live-query 场景覆盖 `ENTER/UPDATE/LEAVE`、`queue gap`、`RESYNC_REQUIRED`、世代 `ACK` 和 `fence drain`。

Agent Loop 与 Workflow EEVDF

7.1 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化、其他 Agent 的消息和 heartbeat。持续轮询会频繁进入内核，也会让大量等待线程占用调度时间。AgentOS 通过有界事件队列和 watch/wait 原语将等待转化为真实睡眠。

每个 Agent 拥有容量 16 的事件队列，其中保留 kernel reserve，并将同一 source 的排队事件限制为 4 条。route 表同样有固定容量。发送 MESSAGE 时，内核检查 MESSAGE_SEND capability、active workflow、source/target route 和 watch，然后将真实 source、target、control id、correlation 和 provenance 写入事件。

agent_wait 先检查已排队事件，再以 thread-generation reservation 安装 waiter 并使线程睡眠。事件入队与 waiter 状态使用相同锁序，确保到达事件能够对应到等待世代。timeout、cancel 和 event consume 均更新 loop state 与 Context。

7.2 Heartbeat 与 LLM correlation

Agent 可设置 heartbeat interval。timer 到期时，内核生成 timer event 并唤醒 Agent，Guest 根据当前状态决定检查文件、发送消息或再次等待。停止 heartbeat 时，generation-bound timer 与 waiter 一并清理。

LLM_REQUEST/LLM_RESPONSE 复用 MESSAGE 和 wait substrate。请求登记 lifecycle、requester/relay 的 PID 与 control id、correlation 和 deadline。correlation 对同一 requester 严格递增，成功入队后才更新。Relay 返回匹配 response 时，内核原子消费 pending 并生成 LLM_DONE。pending 使用 120 秒 kernel-tick TTL，terminal history 区分已消费与超时请求。

7.3 Workflow Credit Domain

资源账户维护 used U 、pending P 、free F 和 $held = U + P + F$ 。当前账户的可用 credit 先在本地状态中转移：

reserve:F→P commit:P→U cancel:P→F release:U→F

账户补充 credit 时检查 global、resource class 和 account hard limit。系统压力、context switch、account close 与 fence 会 trim F 。Workflow Fence 在同一锁中执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、account generation、credit epoch 和向量摘要。

7.4 按 workflow 聚合的 EEVDF

传统线程级公平会使多线程 workflow 获得更多调度候选位。AgentOS 将同一 resource domain 聚合为一个 workflow entity。调度器使用 workflow vruntime 计算 lag, $vruntime \leq global_vtime$ 的实体进入 eligible 集, 再从中选择最早 virtual deadline。latency class 和 event/deadline 可以缩短 service request。一次 dispatch 完成后, 内核按实际 service cycles 更新 workflow vruntime, 睡眠实体的 lag 向 global vtime 衰减。

当只有一个 workflow entity 时, 调度器走 fast path。实体表容量为 4, 其中包含一个 BOOT_SEALED bootstrap participant, 因此同时可纳入三个 fresh workflow。实体信息不完整或容量超出时, 调度器使用原 uCore 路径并增加 fallback 计数。

7.5 唤醒延迟与公平性

一次性实验使用 6 个 fresh QEMU boot, 覆盖并发度 1、2、3、4 的 service window 和 exact wake probe。504 条 probe 直接读取 scheduler trace 中的 ready_age, 其中 425 条为 0 tick, 79 条为 1 tick。uCore tick 为 10 ms, 这些样本表明该负载中的 fresh waiter 都在一个 tick 内获得调度。

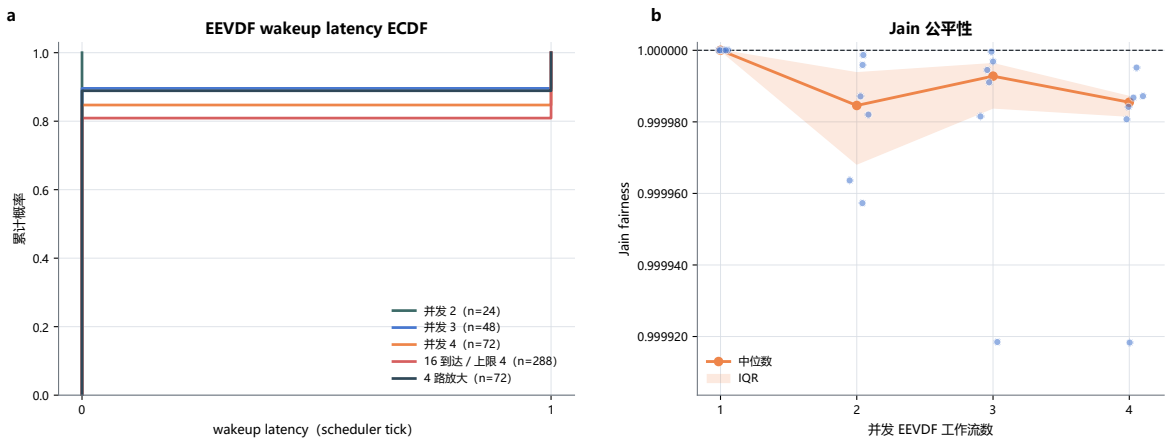


图 7.1: EEVDF 唤醒延迟与公平性

以同一 service window 的 raw cycles 计算 Jain fairness, 并发 1/2/3/4 的中位数分别为 1.000000、0.999985、0.999993 和 0.999985。图 7.1 的 ECDF 保留了每个唤醒探针, 折线则展示不同并发度下 workflow 服务量的均衡程度。

7.6 验证结果

agentloop_ucore 覆盖原子 wait/wakeup、timeout、cancel、heartbeat 和 MESSAGE route; agentsched_ucore 与 agent_eevdf_ucore 覆盖 fast path、实体容量、线程放大、sleep decay、fallback、wake histogram 和 Jain fairness。

AgentOS Console 综合 workflows

8.1 科研恢复场景

AgentOS Console 将六组内核机制组织成一个确定性科研恢复 workflow。labdemo_ucore 创建 Orchestrator、Sentinel、Investigator 和 Recovery，建立 workflow identity、MESSAGE route、Context、metadata 与资源账户。场景登记 prepare/align/analyze/report/archive 五个 stage 及其依赖，然后查询失败节点、读取摘要、跨 Agent 发送恢复请求，完成 align 恢复与 report 更新。

确定性 policy 固定每个阶段的业务工作量，工具结果仍由 AgentOS 内核执行和检查。这样既能连接身份、Context、Live-Query、IPC wait 与恢复写入，也能在不同实现路径间建立等量比较。

8.2 Traversal 与 Indexed 配对

综合场景保留 traversal/compat 和 indexed/native 两条功能等价路径。每个独立 QEMU boot 内按 A/B 或 B/A 顺序运行，两侧输出相同 workload/result fingerprint。核心窗口覆盖查询、恢复写入、fsync 和复核，query block 另行记录纯查询区间。

规范数据集包含 16 个独立 workflow boot，AB/BA 各半。16/16 个 core interval 中 indexed 延迟更低，traversal/indexed core 中位数分别为 34.713 ms 和 13.294 ms，配对加速比中位数为 3.118 倍。

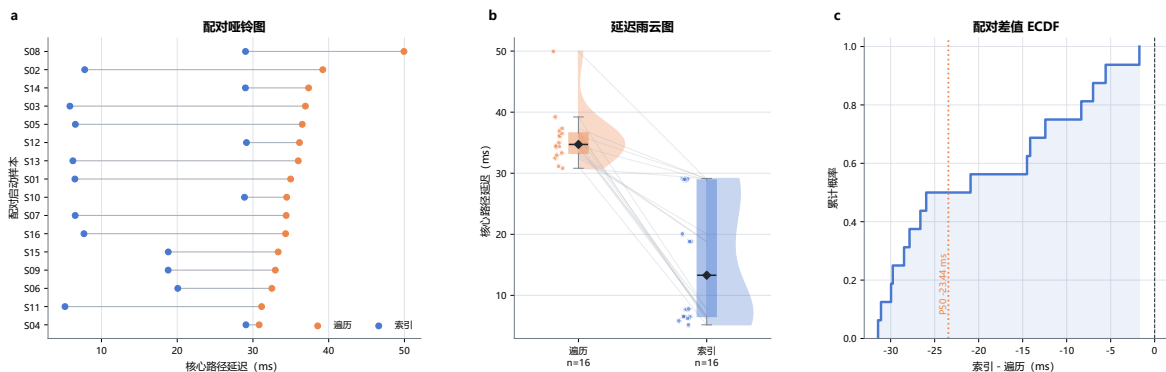


图 8.1: Traversal 与 Indexed Core 配对性能

图8.1的哑铃图逐 boot 连接两条路径，右侧分布图给出各自样本形状，ECDF 则直接展示配对差值。所有配对均落在 indexed 更低的一侧，且保留了不同 boot 的波动。

8.3 Core 与端到端窗口

核心窗口用于观察 AgentOS 数据路径，端到端窗口还包含场景启动、恢复编排、fsync、复核和统计输出。在 E2E 窗口中，indexed 于 3/16 个配对取得更低延迟，indexed-traversal 中位数为 +13.452 ms。这一结果表明，当前整体时间主要受恢复写入、fsync 与复核阶段影响，索引带来的 core 收益还需与这些阶段联合优化。

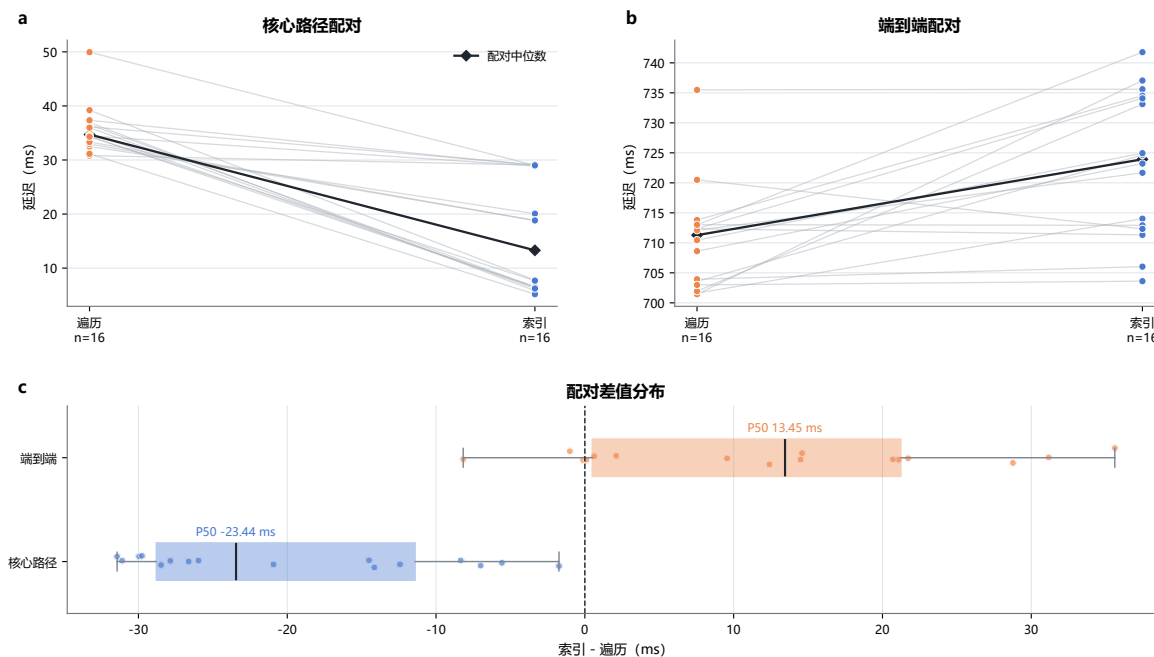


图 8.2: Core 与端到端窗口

AB 与 BA 两个顺序分层的 core speedup 中位数分别为 1.4535 倍和 5.4805 倍。顺序差异使配对设计显得尤其重要：每个 boot 保留双路径原始时间，汇总时再分别展示 AB 和 BA。

8.4 系统结果

综合 workflow 串起 Agent 创建、Context、typed tool、Live-Query、MESSAGE wait、恢复写入和运行记录。输出包含业务 fingerprint、每阶段状态、records examined、核心/E2E 时间和工作量计数。双目标路径另行构建 Plain uCore 与 AgentOS-uCore，在相同用户态合同下比较系统实现。

AgentOS Nexus 多智能体协作平台

9.1 长驻会话与四角色协作

AgentOS Nexus 是建立在 AgentOS-uCore 之上的交互运行时。一个 Nexus session 在同一 workflow 中长驻 Coordinator、System、Research 和 Analyst 四个业务 Agent，每个角色都拥有独立 PID、agent id、control id、role/capability 与 Context。四个角色共享 workflow lifecycle、VFS scope、resource domain 与 EEVDF entity，以工作流作为资源和调度主体。

表 9.1: Nexus 业务角色

产品角色	内核 role	职责	主要 capability
Coordinator	Orchestrator	维护会话、调用模型、创建子任务、验证返回	编排、受控工具、工件 broker
System	Sentinel	读取当前 boot 的进程、Context、文件大小和调度状态	metadata/process/scheduler read
Research	Investigator	读取已登记的本地资料与历史测量工件	content read、query、digest
Analyst	Artifact	合并 System/Research 结果并生成报告	artifact read/write、report publish

另有一个 Relay Agent 负责 QEMU 串口 frame。Host daemon 持有本地 socket、TLS、API key 与 provider JSON adapter，Guest Relay 将 model、controller 和 telemetry frame 分流。业务文件与 artifact 保存在 Guest VFS，Coordinator 依据真实工具返回决定下一步委派。

9.2 一次交互回合

图9.1展示一次从用户目标到最终结果的运行流程。

1. CLI 将新目标作为 turn 提交给 Coordinator。
2. Coordinator 组织有界历史和 role-filtered 工具集，发出 LLM_REQUEST 后进入 WAITING_LLM。
3. Relay 将请求交给 live provider 或 strict replay，响应到达时由 AgentOS 唤醒 Coordinator。
4. 模型需要专门信息时，Coordinator 使用 N1 创建 System、Research 或 Analyst 子任务。
5. Worker 以自己的 identity、Context 和 capability 执行 typed V2 工具，等待期间进入 WAITING_EVENT。

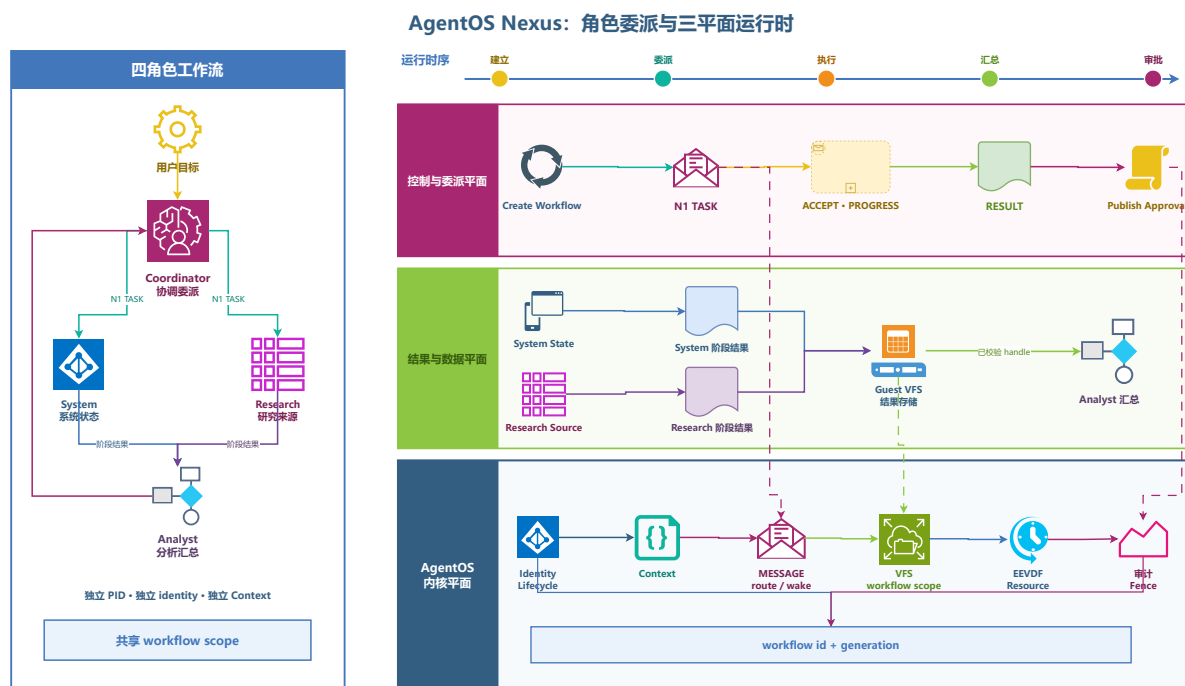


图 9.1: Nexus 四角色委派与三平面运行时

- 短状态经 MESSAGE 返回，长结果写入 workflow-scoped artifact store，Coordinator 验证任务迁移、handle 和 digest。
- 工具结果、结构化失败和审批结果都返回模型，使后续一轮能够继续执行或重新规划。

9.3 N1 子任务协议

N1 是 Guest 用户态的 44-byte canonical little-endian envelope，以 N1：加无填充 base64url 通过 typed V2 send_message 传输。每条任务包含 lifecycle id/generation、task id、parent task id、source/target、deadline、state 和有界 value。状态机包含 TASK_ASSIGN、TASK_ACCEPT、TASK_PROGRESS、TASK_RESULT、TASK_FAILED 和 TASK_CANCEL。

AgentOS 内核对 MESSAGE 层的发送者、接收者、route、scope、provenance、队列和唤醒进行检查，Nexus runtime 处理 N1 decode、transition validation、deadline 和单项在途。较长的 objective、工具结果和报告使用 artifact handle 引用，使控制消息保持紧凑。

9.4 Artifact 数据面

Artifact manifest 绑定 lifecycle、handle generation、kind、task/parent、producer、owner、materializer、permission mask、payload size、source 和 provenance。Guest 以流式 SHA-256 覆盖完整 payload，读取者同时校验 manifest 和 digest。

System 和 Research 在任务终态中返回结构化结果，Coordinator 以 broker 方式将大结果写入 artifact store，并保留 logical producer 与 materializer。Analyst 读取两份上游工件后发

布 report kind。provenance 取各输入的并集，因此报告可以同时保留 file、tool 和 cross-agent 来源。

9.5 动态工具呈现

Nexus 启动时从 AgentOS 内核读取 25 项工具目录，检查 name/ID/schema，再按 product role、kernel role、capability 和 side-effect class 筛选。tool_search 按类别分页返回 rich description。

默认读取面包含 pid_info、ctx_stat、query_process、get_system_status、read_context、query_file、read_file_summary、read_file_digest、dependency_query 和 read_message。delegate_task、read_artifact 与 publish_report 是 Nexus 产品动作，由 Coordinator 组织成 N1、artifact 与 typed V2 操作。

会话保留最近 4 个完整 turn，每个 turn 最多 8 个 model round，更早内容进入结构化 summary。每一轮模型请求都包含当前可见 schema 与上一次稳定工具结果，使工具输出直接改变后续决策。

9.6 调用级审批与取消

publish_report 的审批对象包含 session、turn、request、correlation、tool id、canonical arguments digest、一次性 nonce、issued tick 和 expiry tick。Guest 收到决定后逐字段比较，通过时再执行精确 selector 的 typed V2 artifact_update。strict replay 包含一次拒绝发布，报告 artifact 保持可读，会话继续返回最终结果。

Ctrl-C 取消当前 turn，长驻 Agent 和 QEMU 继续运行；/quit 执行 session close。关闭顺序为停止 observer producer、排空内核观测事件、join observer、关闭 telemetry writer，Relay 在 EOF 后输出 SESSION_CLOSED。

9.7 内核观测面

Nexus observer 从 agent_timeline_wait/read、agent_audit_query、agent_info 和 Context snapshot 生成两类投影。kernel_audit 显示 MESSAGE enqueue/consume 的 lifecycle、identity、source/target、correlation 和 status；kernel_snapshot 显示四个业务 Agent 的 Context sequence、wait/wakeup delta、scheduler dispatch/budget/vruntime 和 capability mask。

controller 在子任务分配和 worker reply 到达时主动 drain 共享 audit cursor，observer thread 使用同一 mutex 访问 cursor。左窗口展示交互与工具结果，右窗口同步显示 AgentOS 对应的消息、等待、唤醒和调度状态。

9.8 运行验证

自动验收使用 strict replay。每条 fixture response 与 Guest 当次产生的 canonical MODEL_REQUEST SHA-256 绑定，并在当前 RV64 镜像的真实 QEMU boot 中执行。黄金场景包含 3 个连续 turn、4 个业务 identity、System 快照、Research stale-handle 失败后重新规划、Analyst 合并报告、一次审批拒绝和完整 session close。live 模式复用同一 Guest、N1、artifact 和 typed tool 路径，在 Host provider adapter 中对接实时模型服务。

功能与性能测试

10.1 测试组织

AgentOS-uCore 的测试按“结构合同、交叉编译、RV64 Guest 场景、Host 运行时、综合性能”五个层次组织。结构合同检查 UAPI 尺寸、偏移与模块依赖；交叉编译将真实 kernel/user object 链接到 RISC-V 镜像；Guest 场景通过系统调用观察 identity、Context、VFS、IPC、resource 与 scheduler 状态；Host 测试覆盖 frame、provider、N1、artifact 和 validator。

表 10.1: 六项核心机制的测试场景

模块	主要程序	关键场景
身份与 Context	agenttrust_ucore, agentscope_ucore, agentfinal_ucore	create/fork/exec/exit、generation reuse、Context snapshot/branch/eviction
结构化工具	agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	typed schema、V3 DAG、retry/cancel、SQ/CQ backpressure/resync
Provenance 与 Fence	agentfinal_ucore, fence/provenance fixtures	来源传播、效果前 gate、critical ring、challenge receipt
Live-Query FS	agentfs_ucore, agentbench_ucore	incarnation、scan/index 一致性、ENTER/UPDATE/LEAVE、resync
Loop 与 EEVDF	agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	atomic wait/wake、timeout、heartbeat、thread amplification、Jain fairness
Console 与 Nexus	labdemo_ucore, agentnexus_ucore	多模块恢复、3 turn/4 role、replan、approval deny、session close

当前版本通过 545 项 Agent 模块与 ABI 合同检查、114 项 Nexus 合同检查以及 93 个 Host 测试程序。strict Nexus replay 在 fresh QEMU boot 中完成 11 个 request digest、3 个 turn、4 个业务角色、失败后重新规划、一次审批拒绝与完整关闭。

10.2 一次性增强实验

为了保留可用于高级图表的逐样本数据，我们在提交 2b14fb1f74b9bd093e6de939a16554620835699e 上完成一次性增强实验。实验使用 30 个 fresh QEMU boot，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都包含来源文件、来源 SHA-256 与原文序号。

表 10.2: 一次性实验组成

实验族	设计	独立单位	样本量
Traversal/Indexed	16 个平衡 AB/BA paired boot	boot	16 个 core 与 E2E 配对
File catalog grid	3 × 4 参数网格, 每格 15 个 inner pair	source boot	180 个查询配对
Batch/Scalar/SQ-CQ	4 boot, 8 组旋转顺序	boot/block	96 条 16-op sequence
Workflow EEVDF	6 boot, 并发度 1–4	boot/window 与 probe	504 条 exact wake probe

同一 boot 中的 inner pair 用于观察稳定工作量下的重复波动, 跨 boot 汇总时使用 source boot 作为独立单位。所有图表直接从 CSV 数据表生成, 使原始顺序、target、workload parameter 和 warning 字段保留在数据中。

10.3 Task 传输延迟

Task 实验使 compact batch、scalar V3 和 SQ/CQ 分别完成 16 次等价 ECHO, 并使用旋转顺序平衡三条路径的先后次序。每条 sequence 记录完整 wall-time, 同时验证 tool/status、Context sequence、Evidence ticket 和 result fingerprint 一致。

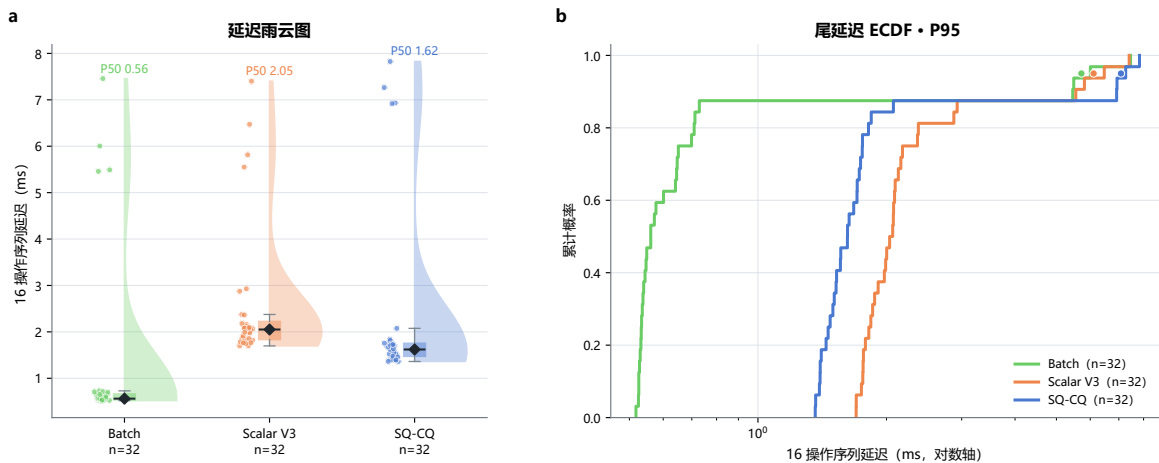


图 10.1: Task 延迟分布

Compact batch、scalar V3 和 SQ/CQ 的 sequence 中位数分别为 561 us、2051 us 和 1620.5 us。Compact batch 在一次 syscall 内顺序执行 16 项操作, 入口开销最低; V3 每次绑定 contract/node/attempt 并执行完整合同检查; SQ/CQ 增加 ring 提交、completion 与 backpressure 语义, 其延迟位于两条同步路径之间。

10.4 内核 I/O 与工作量

性能实验同时记录文件读写、syscall、records examined、Task completion 与 workflow service 等计数。图10.2分为上下两个面板：上半图把 workflow-core 的 bytes_read 和 end-to-end 的全局 I/O 计数分别除以 core-window syscall；下半图按 completed operation 归一化 Agent Task 传输成本。

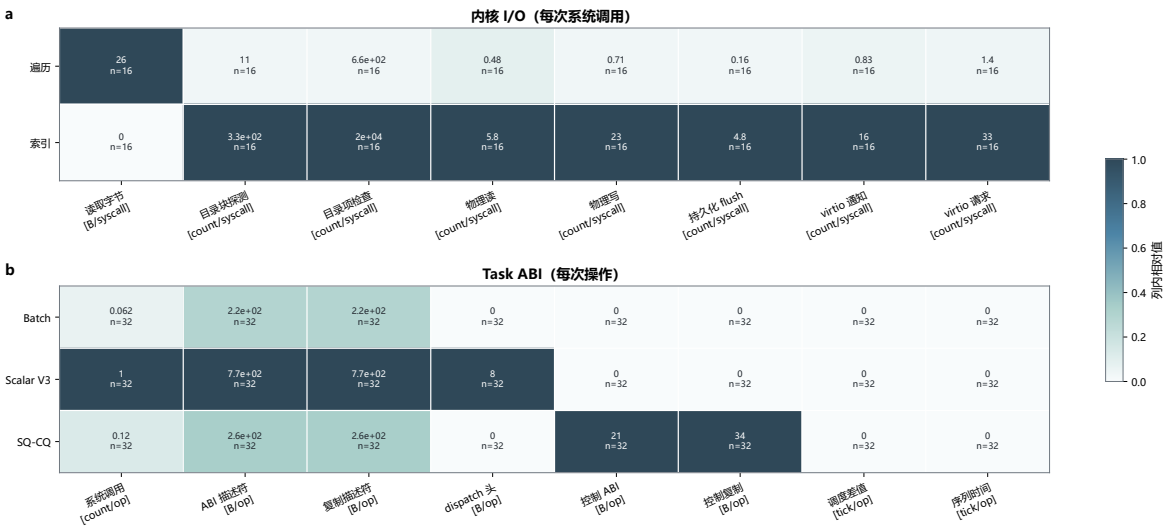


图 10.2: 内核 I/O 与工作量

每一行使用独立量纲与色标，用于比较同一指标在不同路径下的相对变化。

10.5 可重复性

一次性数据集的 manifest 记录源码 commit、工具版本、工作量计划、Guest 与构建哈希，并汇总原始数据、CSV 数据表和图像文件哈希。75 个 inventory 项目全部通过哈希校验。数据用于离线 validate 与 plot，日常回归继续运行常规 Guest 负载。

总结与展望

我们在 RISC-V uCore 中实现了一套系统原生 Agent 运行机制。AgentOS-uCore 以现有进程、页表、VFS、syscall 和 scheduler 为底座，将身份、工具、Context、文件查询、事件等待、资源与调度组织为统一 workflow 主体。六组机制使 Agent 在真实系统效果发生时具有明确的 identity、capability、scope、generation、provenance 和 resource account。

产品的关键进展可以归纳为三点。第一，Agent 已经从应用层标签变成内核可识别的运行主体，普通进程和 Agent 可在同一 uCore 中共存。第二，typed V2/V3、Live-Query、MESSAGE wait、Credit Domain 与 EEVDF 覆盖了 Agent Loop 中从调用、等待到资源服务的主要数据路径。第三，Console 与 Nexus 将内核机制连接到综合科研恢复场景和四角色长驻协作，使产品能够同时支持确定性流程与模型驱动交互。

实验结果展示了 AgentOS 各条路径的性能特征。indexed core path 在 16 个独立配对中全部取得更低延迟，配对加速比中位数为 3.118 倍；12 组 catalog 网格的中位加速比范围为 1.164–2.808 倍；504 条 EEVDF wake probe 全部在 0–1 tick 内得到服务。端到端窗口中 indexed 的中位差值为 +13.452 ms，表明后续优化重点应向恢复写入、fsync 和复核阶段延伸。

下一阶段将沿三条主线继续推进：为 Task Channel 增加业务 payload backend，使 SQ/CQ 承载 Nexus artifact 任务；为 artifact 与 Workflow Fence 增加可选的持久化层，支持跨 boot 恢复；将 workflow EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析多核实体迁移、唤醒与资源记账。这些工作将继续使用 identity、lifecycle 和 workflow resource domain 作为统一主体，使 AgentOS 保持清晰、紧凑的内核结构。

参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础；访问日期：2026-08-11.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024. 访问日期：2026-08-11.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026. 访问日期：2026-08-11.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023. 访问日期：2026-08-11.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026. 访问日期：2026-08-11.
- [6] Jens Axboe. Efficient io with io_uring. https://kernel.dk/io_uring.pdf, 2019. 访问日期：2026-08-11.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999. 访问日期：2026-08-11.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026. 访问日期：2026-08-11.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026. 访问日期：2026-08-11.